

Interview Q&A: pandas

1. What is the purpose of the `read_csv()` function in pandas?

The `read_csv()` function is used to read data from a CSV (Comma Separated Values) file into a pandas DataFrame. It automatically infers data types and handles missing values. You can customize various parameters like delimiter, header row, and encoding.

2. How do you select a column from a pandas DataFrame?

You can select a column using square bracket notation (e.g., `df['column_name']`) or the dot notation (e.g., `df.column_name`). Both methods return a pandas Series containing the selected column's data. You can also select multiple columns using a list.

3. What is the difference between `loc` and `iloc` in pandas?

`loc` is used for label-based indexing, selecting rows and columns by their labels (index and column names). `iloc` is used for integer-based indexing, selecting rows and columns by their integer positions. They are used for selecting data based on their position or label.

4. How do you handle missing values in a pandas DataFrame?

You can use methods like `isnull()` or `isna()` to identify missing values. Then, you can fill them with a specific value using `fillna()`, drop rows with missing values using `dropna()`, or impute them using other techniques like mean or median.

5. Explain the purpose of the `groupby()` method in pandas.

The `groupby()` method groups rows of a DataFrame based on the values in one or more columns. This allows you to perform operations on each group separately, such as calculating aggregate statistics (sum, mean, count) for each group.

6. How do you merge two pandas DataFrames?

You can merge DataFrames using the `merge()` function. It combines rows from two DataFrames based on a common column (or columns). You specify the join type (inner, outer, left, right) and the columns to join on.

7. What is the purpose of the `pivot_table()` function?

`pivot_table()` creates a pivot table from a DataFrame. It reshapes the data by aggregating values based on specified index, columns, and values. This is useful for summarizing data across different categories.

8. How can you perform data cleaning using pandas?

Data cleaning involves handling duplicates, removing outliers, and correcting inconsistencies. Techniques include using `drop_duplicates()`, `replace()`, and applying custom functions to clean data based on specific criteria. You can also use `astype()` to change data types.

9. Explain the concept of `apply()` in pandas.

The `apply()` method applies a function along an axis (rows or columns) of a DataFrame. This is useful for performing complex operations on each row or column, allowing for custom data transformations. It's a powerful tool for data manipulation.

10. How do you read data from a SQL database into a pandas DataFrame?

You can use the `read_sql()` function from pandas, along with a database connection object. This function reads data from a SQL query and returns a DataFrame. You need to provide the database connection, the SQL query, and the DataFrame to be created.

11. What is the difference between `DataFrame.sort_values()` and `DataFrame.sort_index()`?

`sort_values()` sorts the DataFrame based on the values in one or more columns. `sort_index()` sorts the DataFrame based on the index (row labels). They both return a new sorted DataFrame, leaving the original DataFrame unchanged.

12. How can you use pandas to create a time series?

You can create a time series using the `pd.Timestamp` object and the `pd.Series` object. You can then use the `to_datetime()` method to convert strings to datetime objects and create a time series from a list or array of timestamps. You can also use `resample()` to aggregate data over time.

13. What is the primary difference between a Series and a DataFrame in pandas?

A Series is a one-dimensional labeled array, while a DataFrame is a two-dimensional labeled data structure with columns of potentially different types. DataFrames are generally used for tabular data, whereas Series are used for single-column data.

14. How can you create a new DataFrame from a list of dictionaries?

You can use the `pd.DataFrame()` constructor with the list of dictionaries as input. Each dictionary represents a row, and the keys become the column names. This is a straightforward way to initialize a DataFrame from structured data.

15. Explain the purpose of the `fillna()` method in pandas.

The `fillna()` method replaces missing values (NaN) in a DataFrame or Series with a specified value. You can provide a single value, a list of values, or a function to calculate the replacement value. This is crucial for

handling incomplete datasets.

16. What is the difference between `head()` and `tail()` methods?

`head()` returns the first few rows of a DataFrame (default is 5), while `tail()` returns the last few rows. They are used to quickly inspect the beginning and end of a DataFrame to understand its structure and content.

17. How do you perform a pivot operation on a DataFrame?

The `pivot_table()` method reshapes a DataFrame. It allows you to aggregate data based on specified columns and create new columns based on the aggregated values. This is useful for summarizing data and creating new insights.

18. Describe the purpose of the `merge()` function in pandas.

The `merge()` function combines two DataFrames based on a common column or index. You can specify different join types (inner, outer, left, right) to control how rows are matched and combined. This is essential for joining datasets.

19. What is the purpose of the `groupby()` method?

The `groupby()` method groups rows in a DataFrame based on the values in one or more columns. This allows you to perform operations on each group independently, such as calculating aggregate statistics or applying transformations.

20. How can you use `apply()` to apply a custom function to each row or column of a DataFrame?

The `apply()` method allows you to apply a function along an axis (rows or columns). This is useful for performing complex calculations or transformations on the data. You can also use `apply` with `axis=0` for row-wise operations and `axis=1` for column-wise operations.

21. Explain the concept of `inplace=True` in pandas.

The `inplace=True` argument modifies the DataFrame directly without creating a copy. Using `inplace=True` can save memory and execution time, but it also means the original DataFrame is altered. Be cautious when using it as it can lead to unexpected behavior if not handled carefully.

22. How do you create a time series using pandas?

You can create a time series by creating a DataFrame with a datetime index. Then, you can populate the DataFrame with data for each time point. Pandas provides various functions for time series analysis, such as resampling and plotting.

23. What is the purpose of the `drop()` method?

The `drop()` method removes rows or columns from a DataFrame. You can specify the labels of the rows or columns to drop. This is useful for cleaning data, removing irrelevant information, or reducing the size of a DataFrame.

24. How can you use boolean indexing to select rows based on a condition?

Boolean indexing uses a boolean Series to select rows where the condition is `True`. For example, `df[df['column'] > value]` selects rows where the value in the specified column is greater than the given value. This is a powerful way to filter data based on specific criteria.

25. What is the difference between a Series and a DataFrame in pandas?

A Series is a one-dimensional labeled array, while a DataFrame is a two-dimensional labeled data structure with columns of potentially different types. Think of a Series as a single column and a DataFrame as a table.

26. How do you read data from a CSV file into a pandas DataFrame?

You can use the `read_csv()` function. It takes the file path as an argument and automatically infers data types. You can also specify parameters like delimiter, header, and encoding.

27. Explain the purpose of the `fillna()` method.

The `fillna()` method replaces missing values (NaN) in a DataFrame or Series with a specified value. This is crucial for handling incomplete datasets and preventing errors in subsequent analysis. You can specify a constant value or another Series/array.

28. How do you select a column from a DataFrame?

You can select a column using square brackets `[]` with the column name as a string, e.g., `df['column_name']`. Alternatively, you can use the `df.column_name` syntax. Both return a pandas Series.

29. How do you handle missing values in a DataFrame?

You can use methods like `dropna()` to remove rows or columns with missing values, or `fillna()` to replace them with a specified value. Consider the implications of each approach based on the nature of your data.

30. What is the difference between `loc` and `iloc`?

`loc` is used for label-based indexing (selecting rows and columns by their labels), while `iloc` is used for integer-based indexing (selecting rows and columns by their integer positions). They are used for accessing data by their position or label.

31. How do you create a new DataFrame from a list of dictionaries?

You can use the `pd.DataFrame()` constructor and pass the list of dictionaries as an argument. Each dictionary in the list represents a row in the new DataFrame, with keys becoming column names.

32. Explain the purpose of the `apply()` method.

The `apply()` method applies a function along an axis (rows or columns) of a DataFrame. This is useful for performing custom operations on data, such as applying a mathematical function or transforming strings.

33. What is the primary difference between `DataFrame.sort_values()` and `DataFrame.sort_index()`?

`sort_values()` sorts the DataFrame based on the values in one or more columns, while `sort_index()` sorts the DataFrame based on the index (row labels or positions). The index is sorted first, and then the values are sorted within each group.

34. What is the difference between a Series and a DataFrame?

A Series is a one-dimensional labeled array, while a DataFrame is a two-dimensional labeled data structure with columns of potentially different types. Think of a Series as a single column and a DataFrame as a table.

35. How do you read a CSV file into a pandas DataFrame?

Use the `read_csv()` function. You can specify parameters like the delimiter, header row, and data types. For example: `df = pd.read_csv('my_data.csv')`

36. What is the purpose of the `apply()` method in pandas?

The `apply()` method allows you to apply a function along an axis (row or column) of a DataFrame. This is useful for performing custom calculations or transformations on the data.

37. Explain the purpose of the `pivot_table()` function.

The `pivot_table()` function reshapes a DataFrame by aggregating data based on specified columns. It creates a pivot table, allowing you to summarize data in a more compact and organized way.

38. How do you perform boolean indexing to select rows based on a condition?

Use boolean indexing by creating a boolean mask (a Series of `True` and `False` values) based on a condition and then using that mask to select the rows. For example, `df[df['column'] > value]`.

39. What is the purpose of the `merge()` function in pandas?

The `merge()` function combines two DataFrames based on a common column or index. It allows you to join data from different sources into a single DataFrame.

40. How do you sort a pandas DataFrame?

Use the `sort_values()` method to sort by one or more columns. `sort_index()` sorts by the DataFrame's index. You can specify ascending or descending order.

41. How can you select a column from a pandas DataFrame?

You can select a column using square brackets `[]` with the column name as a string, e.g., `df['column_name']`. Alternatively, you can use the dot notation, e.g., `df.column_name`. Both return a pandas Series.

42. What is the purpose of the `merge()` function?

The `merge()` function combines two DataFrames based on a common column or index. It performs a join operation, matching rows based on the specified key. Different types of joins (inner, outer, left, right) are available.

43. Describe the purpose of the `drop()` method.

The `drop()` method removes rows or columns from a DataFrame. You can specify the labels of the rows or columns to drop. It returns a new DataFrame with the specified rows/columns removed, leaving the original DataFrame unchanged unless `inplace=True` is used.

44. Explain how to use the `read_csv()` function with a custom delimiter and specify the encoding.

The `read_csv()` function allows you to read data from a CSV file. You can specify the delimiter using the `sep` parameter (e.g., `sep=';'`) and the encoding using the `encoding` parameter (e.g., `encoding='latin-1'`). This ensures correct parsing of the data.

45. Describe the difference between `DataFrame.fillna()` and `DataFrame.replace()`.

`fillna()` replaces missing values with a specified value, while `replace()` replaces specific values with other values. `fillna()` is primarily for handling missing data, whereas `replace()` is for data transformation and cleaning based on specific value occurrences.

46. How can you use `apply()` to perform a custom calculation on each row of a DataFrame?

The `apply()` method allows you to apply a function along an axis (rows or columns). You can define a function that takes a row or column as input and returns a value. This is useful for complex data transformations and calculations.

47. Explain the purpose of `DataFrame.sort_values()` and how to sort by multiple columns.

The `sort_values()` method sorts a DataFrame based on the values in one or more columns. You can specify the columns to sort by and the sorting order (ascending or descending). Sorting by multiple columns sorts first by the first column, then by the second, and so on.

48. What is the difference between `loc` and `iloc` for selecting data in a DataFrame?

`loc` is used for label-based indexing, selecting rows and columns by their labels (index and column names). `iloc` is used for integer-based indexing, selecting rows and columns by their integer positions. They are distinct methods for accessing data based on different criteria.

49. Describe a scenario where you would use `groupby()` to aggregate data.

You would use `groupby()` to group rows based on one or more columns and then apply aggregation functions (like `sum()`, `mean()`, `count()`) to calculate summary statistics for each group. This is useful for analyzing data by category or grouping.

50. How can you use boolean indexing to filter rows in a DataFrame?

Boolean indexing uses a boolean Series to select rows where the condition is `True`. For example, `df[df['column'] > value]` selects rows where the value in the specified column is greater than the given value. This is a powerful way to filter data based on specific criteria.